Zadání projektu

Obecné pokyny

- Cvičícím pro zde popsané varianty je Ondřej Ondryáš. Konzultace poskytuji primárně na Discordu (`@i_am_leo_`) nebo přes e-mail (`iondryas@fit.vut.cz`).
- Projekty můžete řešit individuálně nebo ve dvojicích. V případě řešení ve dvojici očekávám propracovanější řešení a vysvětelení přínosu obou řešících. Řešíte-li ve dvojici, na zadání v IS VUT se přihlašují **oba** řešící, odevzdat stačí jednou.
- Projekt bude hodnocen především na základě **demonstrace cvičícímu** (online nebo prezenčně). Demonstrace budou probíhat **cca od 8. do 19. 12.**, o možnostech výběru termínu vám ještě zašlu zprávu. Po dohodě se mnou je možná i pozdější demonstrace, v každém případě by měla proběhnout nejpozději **4. 1. 2026**. Nejpozději **24 hodin** před zvoleným termínem demonstrace **odevzdejte** do IS archiv s řešením projektu. **Neřiďte se datem odevzdání v IS**, ale tímto bodem. Pokud už nebudete mít možnost odevzdat do ISu, pošlete mi řešení e-mailem.
- Bližší požadavky obsah odevzdávaného archivu mohou být uvedeny u jednotlivých variant zadání. Pokud není řečeno jinak, není pro archiv vyžadována žádná konkrétní struktura, neodevzdávejte však nezajímavý odpad typu adresář `.git`.
- Součástí všech zadání je jak programování, tak konfigurace nějaké infrastruktury. Hlavním cílem projektu je, abyste si některé z přednášených konceptů/technologií „osahali“. Není tedy vůbec nutné, aby vaše řešení dosahovalo produkční kvality, ale pokuste se aspoň o nějakou míru přehlednosti a přenositelnosti (*já to spíš pouštět nebudu, ale dělejte, jako by to někdo jiný u sebe mohl chtít spustit*).

  Za téměř povinné však považujte odevzdání funkční konfiguraci(/-e) pro Docker Compose, Vagrant, Ansible, Helm, Terraform, Nix nebo podobné platformy/nástroje či jejich kombinace, které potenciální nasazení vašeho řešení umožní.
- Zadání jsou záměrně napsána obecně a bez mnoha konkrétních požadavků – záleží na vás, jaké zdroje dat, přístupy a technologie zvolíte. Nebojte se, že byste udělali „příliš málo práce“, pro plný počet bodů stačí (dobře odprezentované) řešení, které splňuje všechny *požadavky* konkrétní varianty zadání.
- V případě nedostatku inspirace vám jistě pomohou velké jazykové modely, nicméně při samotném řešení je prosím (pro vlastní dobro) užívejte opravdu pouze pro podporu při řešení drobných, konkrétních problémů. *High-level* přístup k řešení zkuste vymyslet sami.
- Pokud vám některé ze zadání připomíná nějaký praktický problém, který právě řešíte (nebo jste v nedávné minulosti vyřešili), můžete se mi ozvat – budu rád, když pro vás bude mít práce na projektu i nějaké skutečné využití.
- Pokud pro nějakou úlohu nenajdete vhodný „skutečný“ zdroj proudových dat, můžete využít i statický, ale dostatečně objemný zdroj dat (např. rozsáhlý záznam o síťových tocích). V každém případě musí být řešení navrženo tak, aby do něj šel real-time zdroj „pustit“ bez dalších koncepčních úprav („streaming-first“ přístup).

Různé zdroje (real-time) dat

Níže najdete odkazy na různé zdroje (či další seznamy zdrojů), která můžete (ale nemusíte) využít v některých variantách zadání. Je vhodné vybrat si zdroj, který produkuje nová data relativně často (řádově max. minuty).

- Pražská datová platforma
- Datový portál města Brna (pozor, zde je mix statických i pravidelně aktualizovaných dat, musíte si datový zdroj otevřít, pak kliknout na *View Full Details* a z popisu vyčíst, jestli a jak často se zdroj aktualizuje)
- Pohyb vozidel IDS JMK
- ČHMÚ – stav ovzduší
- `https://github.com/ColinEberhardt/awesome-public-streaming-datasets`
- `https://github.com/apgiorgi/awesome-streaming-data-sources`
- `https://ably.com/blog/10-realtime-data-sources-you-wont-believe-are-free`

Seznam variant

1 ETL pipeline **3**

2 Detekce anomálií (např. DDoS útoků) **4**

3 Materializované pohledy na data o provozu sítě **5**

4 Integrace služeb s využitím zasílání zpráv **6**

5 Demonstrace problémů v Apache Flink **7**

6 Zpracování dat v Microsoft Azure **8**

1 ETL pipeline

Cíl: Vytvořte ETL (extract–transform–load) pipeline, která v reálném čase načítá data z alespoň dvou zvolených zdrojů dat, které vhodně spojuje a transformuje, načež výsledek ukládá do databáze.

Požadavky:

- Data načítejte do Apache Kafka.
- Využijte alespoň tři *topics* (pravděpodobně dvě pro „surová“ data, jedno pro „zpracovaná“ data). Nastavte u nich vhodný *partitioning* a vhodnou politiku retence dat (s ohledem na charakter datových proudů).
- Zpracovaná data exportujte do alespoň jednoho externího databázového systému. Můžete zvolit relační i nerelační či post-relační databázi. Měla by mít vhodný formát (např. při využití relační databáze dbejte na normalizaci schématu).
- Celý systém by měl zaručovat „exactly-once“ sémantiku zpracování dat.
- Implementujte na datech nějakou ne zcela triviální transformaci (například nějakou agregaci s vhodným použitím oken).

Doporučení:

- Pro import a export dat využijte Kafka Connect.
- Pro zpracování dat využijte Apache Flink, případně Kafka Streams.
- Využít můžete také nástroj Flink CDC.

2 Detekce anomálií (např. DDoS útoků)

Cíl: Využijte Apache Flink k netriviální detekci anomálií v nějakém real-time zdroji dat.

Požadavky:

- Použijte proudový zdroj síťových událostí (např. NetFlow/IPFIX) nebo syntetický generátor s parametrizovanými scénáři útoků i normálního stavu.
- Implementujte pipeline platformy Apache Flink s zpracováním podle event time a ošetřením opožděných dat (vč. správné konfigurace *watermarks*).
- Naimplementujte alespoň dvě nezávislá detekční pravidla. Příklady: nával nových zdrojových IP na jeden cíl, burst SYN paketů, rychlé skenování portů, náhlá změna poměru pps/bps/flows.
- Zvolte vhodnou formu výstupu (detekce by typicky měla vést na vystavení nějakého *alertu*). Součástí výstupu by mělo být vysvětlení anomálie (počet, poměr, ...).
- Veškeré prahy a okna by měly být konfigurovatelné.
- Pro demonstraci připravte alespoň dva scénáře (např. mnoho požadvaků na jeden port + horizontální sken sítě) a ukažte jejich zachycení. Dále demonstrujte chování na běžném provozu.

Doporučení

- Využít můžete také knihovnu Flink CEP.
- Alerty můžete ukládat do databáze, pro jejich získávání můžete vytvořit např. jednoduché REST API, případně si vyzkoušejte použití aplikace Grafana.

3 Materializované pohledy na data o provozu sítě

Cíl: Použijte Apache Flink SQL pro výpočet statistik o síťovém provozu (nejspíše na základě toků dat z NetFlow/IPFIX).

Požadavky:

- Definujte schéma síťových toků (NetFlow/IPFIX či obdobné) jako tabulku pro Flink SQL (nezapomeňte na správnou definici event time, *watermarks*).
- Implementujte alespoň 5 materializovaných pohledů nad proudem, které výpočet nějakých zajímavých metrik. Příklady:
  - „Top talkers“ – největší zdroje/ cíle podle bps/pps/flows,
  - množství provozu per L4 protokol/port, a to absolutní i relativní počet,
  - počet různých zdrojových IP na službu/port,
  - agregace po síťových prefixech (např. /24, /48) – objem a počet toků.
- Zvolte sledované metriky tak, abyste smysluplně využili alespoň dva druhy oken. Zahrnout můžete i nějakou globální metriku (pozor však, abyste nepoužívali potenciálně neomezené množství paměti!).
- U každého pohledu jasně definujte primární klíč a okno (tumbling/sliding, šířka a posun).
- Popište, jak řešíte zpožděná data.
- Bonus: pokud bude sledovat nějaký počet různých (unikátních) prvků (*count-distinct problem*), implementujte algoritmus HyperLogLog.

4 Integrace služeb s využitím zasílání zpráv

Cíl: Integrujte nějaké existující nebo vlastní (mikro)služby s využitím zasílání zpráv a Apache Kafka jako centrální sběrnice. Inspirujte se koncepty „Enterprise Service Bus“.

Požadavky:

- Integrujte alespoň tři (zvolené) služby, a to s využitím komunikace zasíláním zpráv se sběrnicí. Celé řešení by mělo konceptuálně následovat principy ESB/MOM a pro zasílání zpráv využívat Apache Kafka.
- Můžete vytvořit adaptér pro stávající rozhraní (např. HTTP REST API) vybraných služeb nebo podporu doimplementovat přímo do nich.
- Implementujte mikroslužbu, která zajistí směrování zpráv, a alespoň dvě další pomocné komponenty architektury ESB (např. validátory, *enrichment*, transformace).
- Cílem není naimplementovat kompletní/produkční ESB – **není** nutné studovat a využít obsáhlé standardy, zprávy ve formátu XML a podobně.
- **Alternativně** můžete naopak zprovoznit nějaké produkční řešení ESB (nejlépe volně dostupné, např. Apache Camel). V tomto případě nebudete implementovat vlastní směrování a pravděpodobně nevyužijete Apache Kafka.

5 Demonstrace problémů v Apache Flink

Cíl: Vytvořte libovolnou pipeline platformy Apache Flink (pravděpodobně se syntetickým zdrojem dat), na které prakticky demonstrujete problémy nerovnoměrného rozložení dat napříč operátory (*data skew*), pomalých cílových uzlů (*slow sinks*) a propagace *backpressure*, a zpracování zpráv, které přijdou ve špatném pořadí (*out-of-order*) nebo se zpožděním (*lateness*). Diskutujte příčiny a možná řešení těchto problémů.

6 Zpracování dat v Microsoft Azure

Cíl: Navrhněte a zprovozněte pipeline pro načítání, proudové zpracování a ukládání dat s využitím výhradně nativních služeb Azure. Zaměřte se na event-time zpracování, toleranci zpožděných zpráv a reprodukovatelné nasazení.

Požadavky:

- Zvolte alespoň jeden real-time zdroj (nebo syntetický generátor) a napojte jej do Azure Event Hubs. Vytvořte alespoň 2 *consumer groups* (např. pro zpracování a pro ladění/monitoring).
- Implementujte libovolnou logiku zpracování dat v Azure Stream Analytics (ASA) nebo v Azure Functions. Zpracování musí zahrnovat minimálně dvě odlišné operace (okna, joins, deduplikace se zachováním idempotence, jednoduchou detekci anomálií, ...).
- Uložte výsledek např. do Azure Cosmos DB.
- Připravte vhodný způsob zobrazování výsledků, např. Azure SQL dotazy.
- O procesu sbírejte alespoň nějaké základní metriky pomocí Azure Monitor.
- Demonstrujte reprodukovatelnost vytvořením konfigurace pro Azure Resource Maanger (ARM template nebo Bicep), která celé řešení nasadí.

Doporučení:

- Po přihlášení svým VUT účtem zde získáte 100 dolarů kreditu v Azure.